

Experiment 07– Apply Genetic Algorithm on given problem.

Learning Objective: Students should be able to solve an optimization problem using Genetic Algorithm.

Tools: Python under Windows or Linux environment

Theory: Study and implement Genetic Algorithms for optimization problems.

In a genetic algorithm, a population of candidate solutions (called individuals, creatures, or phenotypes) to an optimization problem is evolved toward better solutions. Each candidate solution has a set of properties (its chromosomes or genotype) which can be mutated and altered; traditionally, solutions are represented in binary as strings of 0s and 1s, but other encodings are also possible.

The evolution usually starts from a population of randomly generated individuals, and is an iterative process, with the population in each iteration called a generation. In each generation, the fitness of every individual in the population is evaluated; the fitness is usually the value of the objective function in the optimization problem being solved. The more fit individuals are stochastically selected from the current population, and each individual's genome is modified (recombined and possibly randomly mutated) to form a new generation. The new generation of candidate solutions is then used in the next iteration of the algorithm. Commonly, the algorithm terminates when either a maximum number of generations has been produced, or a satisfactory fitness level has been reached for the population.

A typical genetic algorithm requires:

1. a genetic representation of the solution domain,
2. a fitness function to evaluate the solution domain.

A standard representation of each candidate solution is as an array of bits. Arrays of other types and structures can be used in essentially the same way. The main property that makes these genetic representations convenient is that their parts are easily aligned due to their fixed size, which facilitates simple crossover operations. Variable length representations may also be used, but crossover implementation is more complex in this case. Tree-like representations are explored in genetic programming and graph-form representations are explored in evolutionary programming; a mix of both linear chromosomes and trees is explored in gene expression programming.

Once the genetic representation and the fitness function are defined, a GA proceeds to initialize a population of solutions and then to improve it through repetitive application of the mutation, crossover, inversion and selection operators.



ALGORITHM: Genetic Algorithm

ADVANTAGES:

1.
2.

DISADVANTAGES:

1.
2.



APPLICATIONS:

1.

2.

Implementation:

Result and Discussion:

Learning Outcomes: The student should have the ability to

LO4.1 identify the need of optimization

LO4.2 define Genetic algorithm

LO4.3 apply GA to an optimization problem

LO4.4 identify advantages and disadvantages of GA

Course Outcomes: Apply various AI search algorithms uninformed, informed, local, adversarial and backtracking search algorithms and constraint satisfaction problems to real-world problems.

Conclusion:

Viva Questions:

1. Explain optimization algorithms apart from GA.
2. What kind of problems are solved through optimization algorithms?
3. State Genetic Operators?

For Faculty Use

Correction Parameters	Formative Assessment [40%]	Timely completion of Practical [40%]	Attendance / Learning Attitude [20%]	
Marks Obtained				